SWISS PYTHON SUMMIT 2026

# Profiling Python

FROM CPROFILE TO SAMPLING

# One program

- DEMO_WORKLOAD.PY

```
def tiny_add(x, y):
    return x + y

def many_tiny_calls(n):          # 10 million tiny calls
    total = 0
    for i in range(n):
        total = tiny_add(total, i)
    return total

def cpu_work(n):                 # 10 million square roots
    total = 0.0
    for i in range(1, n):
        total += math.sqrt(i)
    return total

def io_wait():                   # waiting
    time.sleep(1.0)
```

- QUESTION

## Which one is slowest?

# Without a profiler

| FUNCTION | TIME | SHARE |
|---|---|---|
| many_tiny_calls | 0.57 s | 25% |
| cpu_work | 0.69 s | 31% |
| io_wait | 1.00 s | 44% |

NO PROFILER · TIME.PERF_COUNTER() · PYTHON 3.15.0

REMEMBER

44%

IO_WAIT · SHARE OF RUNTIME

# Kinds of profilers

| TYPE | ANSWERS | EXAMPLES | TODAY |
|---|---|---|---|
| Tracing DETERMINISTIC | Exact calls, call graph | cProfile | ● |
| Sampling STATISTICAL | Where does time go? | Tachyon · py-spy · austin | ● |
| Line-level | Which line is slow? | line_profiler · --heatmap | ○ |
| Memory | Where are allocations? | tracemalloc · memray | ○ |
| Benchmark | Is A faster than B? | timeit · pyperf | ○ |

# How tracing works

- A hook runs on **every** call and return
- Python functions and built-ins like `math.sqrt`
- Result: **exact** call counts and a full call graph

# Demo: tracing

```
$ python -m profiling.tracing -s tottime demo_workload.py | head -n 14
         20000161 function calls (20000154 primitive calls) in 6.240 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    1.767    1.767    2.728    2.728 demo_workload.py:13(many_tiny_calls)
        1    1.358    1.358    2.512    2.512 demo_workload.py:20(cpu_work)
  9999999    1.153    0.000    1.153    0.000 {built-in method math.sqrt}
        1    1.000    1.000    1.000    1.000 {built-in method time.sleep}
 10000000    0.961    0.000    0.961    0.000 demo_workload.py:9(tiny_add)
        2    0.000    0.000    0.000    0.000 {built-in method _imp.create_builtin}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
      2/1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1333(_find_and_load)
        1    0.000    0.000    6.240    6.240 {built-in method builtins.exec}
$
```

PYTHON 3.15: CPROFILE IS ALSO AVAILABLE AS PROFILING.TRACING

# Before and after

| FUNCTION | NO PROFILER | CPROFILE | SLOWDOWN |
|---|---|---|---|
| many_tiny_calls | 0.57 s | 2.73 s | 4.8× |
| cpu_work | 0.69 s | 2.51 s | 3.6× |
| io_wait | 1.00 s | 1.00 s | 1.0× |
| Total | 2.26 s | 6.24 s | 2.8× |



# The ranking flips

● REALITY

## 44%

| | | |
|---|---|---|
| 01 | io_wait | 44% |
| 02 | cpu_work | 31% |
| 03 | many_tiny_calls | 25% |

● UNDER CPROFILE

## 16%

| | | |
|---|---|---|
| 01 | many_tiny_calls | 44% |
| 02 | cpu_work | 40% |
| 03 | io_wait | 16% |

You would focus on the loops, and think waiting is a small cost.

# Why?

- Every call pays a tracing cost
- ~20 million calls → big cost
- `sleep` is **one** call → almost no cost

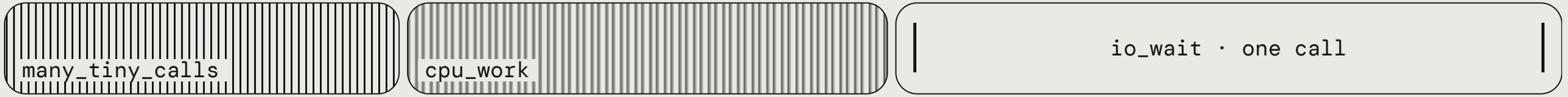
> cProfile isn't lying.
> It measures a program that tracing made slower.

# Diary vs photos

| | |
|---|---|
| TRACING | write down every step |
| SAMPLING | photo of the call stack every millisecond |

TRACING · A MARK FOR EVERY CALL

CALLS 20,000,161




SAMPLING · EACH TICK = ONE STACK SNAPSHOT

SAMPLES 2,288



# The trade-offs

- **Upside:** reads the stack from outside → almost no overhead
- Estimates, not measurements
- Results vary slightly between runs
- Very short functions can be missed
- **No call counts**
- Runs under ~1 s → too few samples

# Two runs, two answers

RUN 1

| | | |
|---|---|---|
| 01 | io_wait | 47.9% |
| 02 | cpu_work | 33.2% |
| 03 | many_tiny_calls | 14.1% |
| 04 | tiny_add | 4.8% |

RUN 2

| | | |
|---|---|---|
| 01 | io_wait | 46.5% |
| 02 | cpu_work | 31.7% |
| 03 | many_tiny_calls | 15.5% |
| 04 | tiny_add | 6.2% |

## Read the pattern, not the decimals.

SAME COMMAND, TWO RUNS · SHARE OF SAMPLES WHERE THE FUNCTION WAS RUNNING

# 10 million calls — or one?

- TRACING

10,000,000

TINY_ADD · EXACT CALL COUNT

- SAMPLING

≈ 5%

TINY_ADD · SHARE OF SAMPLES · NO COUNT

Sampling shows time, not counts.

# The ecosystem proved it

- PY-SPY

Rust

READS THE STACK
FROM OUTSIDE

- AUSTIN

C

READS THE STACK
FROM OUTSIDE

- TACHYON

stdlib

PYTHON 3.15

The stdlib had profiling. It didn't have sampling.

# profiling.sampling

```
python -m profiling.sampling run demo_workload.py
```

| | |
|---|---|
| COMMANDS | run · attach · dump · replay |
| OUTPUTS | pstats · flamegraph · heatmap · gecko · collapsed · binary |

# From stacks to flame graph

```
main;io_wait                    1000
main;cpu_work                    760
main;many_tiny_calls             545
main;many_tiny_calls;tiny_add    191
```



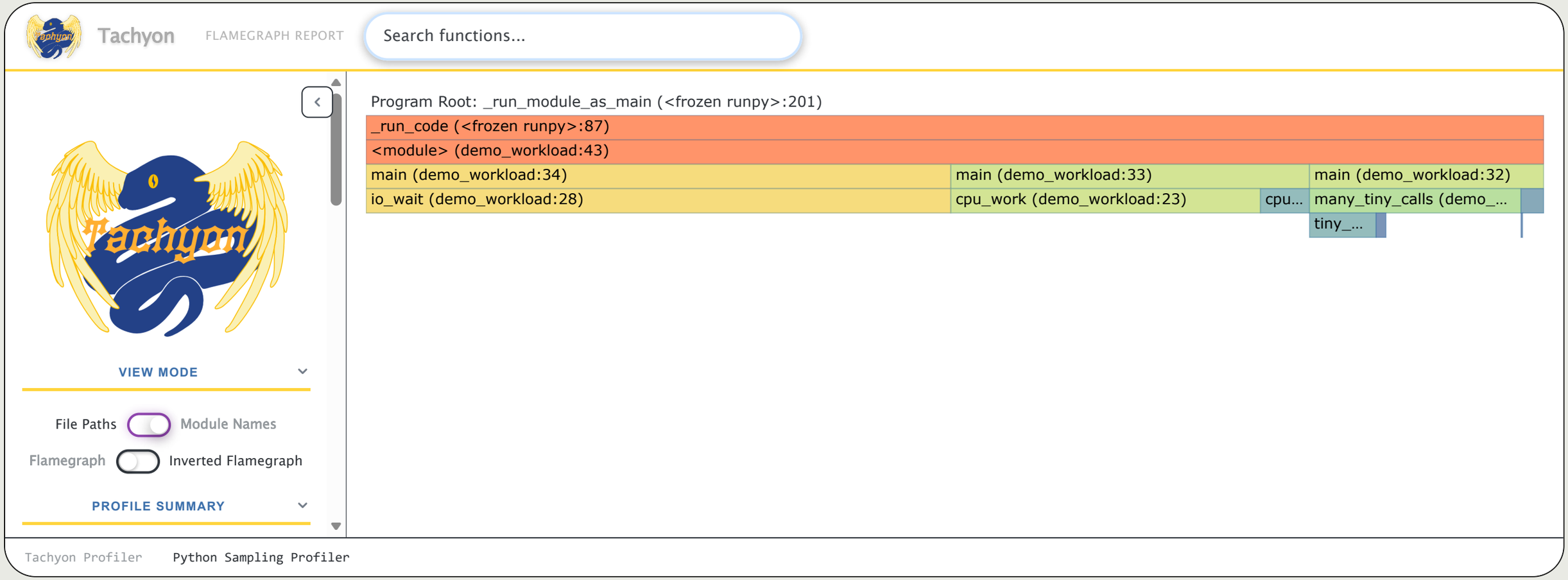

REAL COUNTS FROM ONE RUN · --COLLAPSED OUTPUT, SIMPLIFIED

# Reading a flame graph

| | |
|---|---|
| WIDTH | share of samples |
| HEIGHT | stack depth |
| X-AXIS | not time |
| WIDE BOX, NOTHING ABOVE | time spent in that function itself |

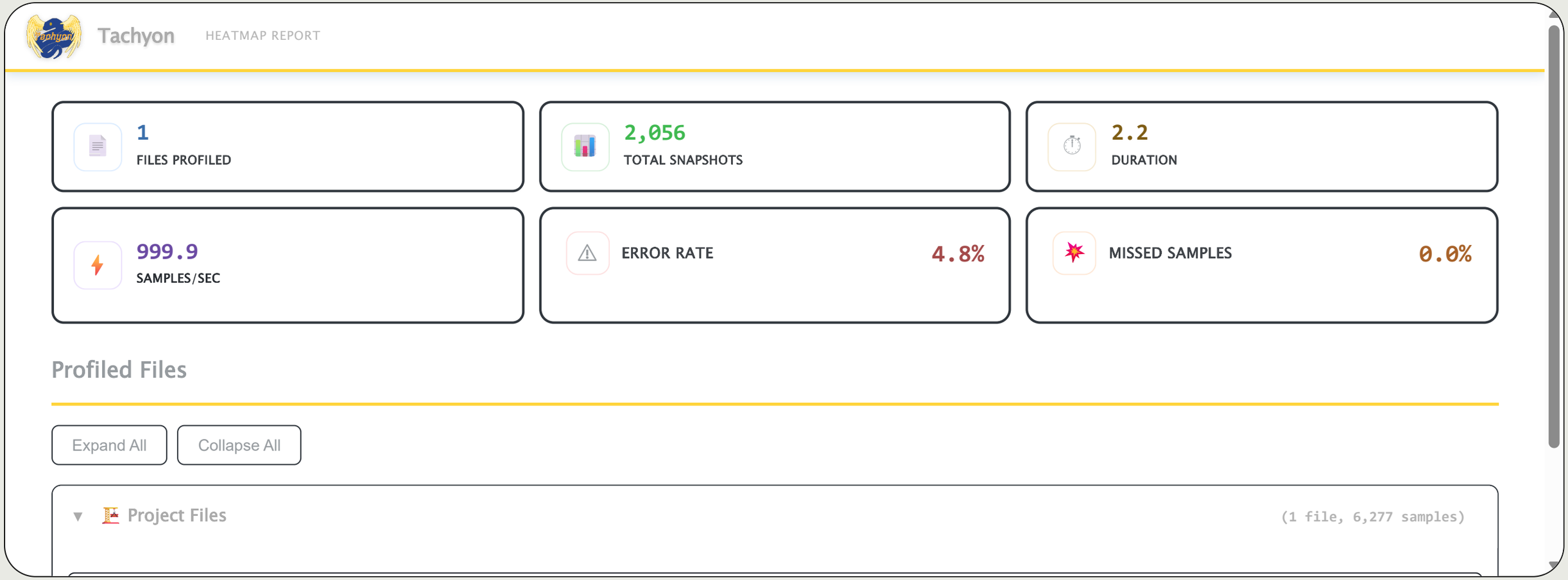
# Demo: flame graph

```
$ python -m profiling.sampling run --flamegraph demo_workload.py
```



# Demo: heatmap

```
$ python -m profiling.sampling run --heatmap demo_workload.py
```



# Two questions

- `--MODE=WALL`

Where does time pass, including waiting?

- `--MODE=CPU`

Where are CPU cycles spent?

`--MODE=WALL · EVERY TICK IS KEPT`



# Demo: wall mode

```
$ python -m profiling.sampling run demo_workload.py
Captured 2,288 samples in 2.29 seconds
Sample rate: 999.91 samples/sec
Error rate: 5.64
Profile Stats:
       nsamples   sample%   tottime (s)    cumul%   cumtime (s)  filename:lineno(function)
      1000/1000      46.4         1.000      46.4         1.000  demo_workload.py:28(io_wait)
        602/602      27.9         0.602      27.9         0.602  demo_workload.py:23(cpu_work)
        284/418      13.2         0.284      19.4         0.418  demo_workload.py:16(many_tiny_calls
)
        100/100       4.6         0.100       4.6         0.100  demo_workload.py:22(cpu_work)
          95/95       4.4         0.095       4.4         0.095  demo_workload.py:10(tiny_add)
          39/39       1.8         0.039       1.8         0.039  demo_workload.py:9(tiny_add)
          35/35       1.6         0.035       1.6         0.035  demo_workload.py:15(many_tiny_calls
)
$
```

# Demo: CPU mode

```
$ python -m profiling.sampling run --mode=cpu -r 100 demo_workload.py
Captured 239 samples in 2.39 seconds
Sample rate: 100.00 samples/sec
Error rate: 7.53
Profile Stats:
       nsamples   sample%  tottime (ms)    cumul%   cumtime (s)  filename:lineno(function)
          66/66      55.9       660.000      55.9         0.660  demo_workload.py:23(cpu_work)
          23/38      19.5       230.000      32.2         0.380  demo_workload.py:16(many_tiny_calls
)
          10/10       8.5       100.000       8.5         0.100  demo_workload.py:22(cpu_work)
            9/9       7.6        90.000       7.6         0.090  demo_workload.py:10(tiny_add)
            6/6       5.1        60.000       5.1         0.060  demo_workload.py:9(tiny_add)
            4/4       3.4        40.000       3.4         0.040  demo_workload.py:15(many_tiny_calls
)
           0/42       0.0         0.000      35.6         0.420  demo_workload.py:32(main)
$
```

# Reading both

| | |
|---|---|
| HIGH IN BOTH | compute hotspot |
| HIGH IN WALL ONLY | waiting · I/O, sleep, locks |

- WALL
  EVERYTHING
- CPU
  ON CPU
- GIL
  HOLDING THE GIL
- EXCEPTION
  EXCEPTION ACTIVE

# The sleep, four ways

- REALITY

44%

- CPROFILE

16%

- TACHYON WALL

46%

- TACHYON CPU

0%

Same program. Different profilers. Different truths.

# Which profiler?

| QUESTION | TOOL |
| --- | --- |
| How many times is X called? | Tracing |
| Where does time pass, including waiting? | Tachyon --mode=wall |
| Where are CPU cycles spent? | Tachyon --mode=cpu |
| What is production doing, without restart? | attach · dump · py-spy · austin |
| Is A faster than B? | timeit · pyperf |

# Three rules

| | | |
|---|---|---|
| 01 | Need exact call counts? | Tracing |
| 02 | Need representative hotspots? | Sampling |
| 03 | Comparing small speed differences? | Benchmark |

# Thank you

| | |
|---|---|
| TACHYON | docs.python.org/3.15/library/profiling.sampling.html |
| PY-SPY | github.com/benfred/py-spy |
| AUSTIN | github.com/P403n1x87/austin |
| FLAME GRAPHS | brendangregg.com/flamegraphs.html |
| CONTACT | |

